Light

Data Representation — Numbers

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

Assessment Hub | Notes / Textbook | Long-Answer Practice

Assessment Hub | Long-Answer | Notes Index

What you must know | Toolkit | 1. Binary and denary

2. Hexadecimal and two's complement | 3. Binary arithmetic

Worked examples | Quick reference | Typical pitfalls

Edexcel GCSE Computer Science — **Data Representation: Numbers (1CP2/01 Topic 2: Data)**. This topic covers how computers represent numbers using binary (base-2), conversions between binary, denary (base-10) and hexadecimal (base-16), representing negative numbers using two's complement, and performing binary arithmetic including addition, overflow detection, and binary shifts.

Exam questions typically ask you to **convert** between number systems, **add** binary numbers, **identify** overflow, **apply** binary shifts, and **represent** negative numbers in two's complement. The 2024 Paper 1 included multi-step conversion questions and binary addition with overflow detection. Expect questions worth 2—6 marks that require clear working and understanding of place values.

Why number representation matters

Computers can only work with binary — patterns of 0s and 1s representing electrical signals (off/on). Understanding how numbers are represented in binary helps you:

- **Understand computer limitations:** Why integers have maximum values and why overflow occurs
- **Debug programs:** Recognising when calculations exceed data type limits

- **Work with low-level data:** Understanding memory addresses, colour codes, and file formats
- **Appreciate hexadecimal:** Why programmers use hex as a shorthand for binary
- **Succeed in the exam:** Number conversions and binary arithmetic appear in Paper 1 every year, typically worth 6—12 marks
  In the exam, you must show clear working for all conversions and calculations. No calculator is allowed, so you need to be confident with powers of 2.

What you must know

This checklist covers the key exam-visible concepts from the Edexcel specification (2.1). You should be able to perform each skill without a calculator.

- **Convert** positive integers between binary (base-2) and denary (base-10) for values up to 255 (8-bit)
- **Convert** between binary and hexadecimal (base-16), and between hexadecimal and denary
- **Explain** why hexadecimal is used as a shorthand for binary in computing
- **Represent** negative integers using 8-bit two's complement
- **Add** two 8-bit binary integers and identify when overflow has occurred
- **Apply** binary shifts (left and right) and explain their effect on the value

> **Specification link:** This topic covers Edexcel 1CP2 specification section 2.1 — "understand how computers represent and manipulate numbers: unsigned integers, signed integers (two's complement), addition of binary integers, binary shifts."

Toolkit

These are the essential reference tables and patterns you need for exam questions.

Powers of 2 (memorise these!)

Power	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8
Value	1	2	4	8	16	32	64	128	256

8-bit binary place values

128	64	32	16	8	4	2	1
				2^7			$2^6 2^5 2^4 2^3 2^2 2^1 2^0$

8-bit unsigned range: 0 to 255 ($2^8 - 1$)

Hexadecimal digits

Denary	0	1	2	3	4	5	6	7	8	
Hex	0	1	2	3	4	5	6	7	8	9

Denary	0	1	2	3	4	5	6	7	8	
Binary	0000	0001	0010	0011	0100	0101	0110	0111	1000	1

Two's complement (8-bit) place values

-128	64	32	16	8	4	2	1

8-bit two's complement range: -128 to +127
The leftmost bit (MSB) has a **negative** place value in two's complement.

Binary shift effects

Shift	Direction	Effect on value	Bits lost	Bits added
Left shift by 1	← (left)	Multiply by 2	MSB falls off left	0 added on right
Left shift by 2	← (left)	Multiply by 4 (2^2)	2 MSBs fall off	00 added on right
Right shift by 1	→ (right)	Divide by 2 (integer division)	LSB falls off right	0 added on left
Right shift by 2	→ (right)	Divide by 4 (2^2)	2 LSBs fall off	00 added on left

Exam tip: Left shift by n = multiply by 2^n. Right shift by n = integer divide by 2^n. A right shift of an odd number loses the remainder (e.g., 7 >> 1 = 3, not 3.5).

1. Binary and denary

Binary (base-2) uses only two digits: 0 and 1. **Denary** (base-10) is the number system we use every day with digits 0—9. Converting between these systems is a fundamental skill tested in every exam.

1.0 Why computers use binary

Computers use binary because their electronic circuits have two stable states: on (high voltage) and off (low voltage). These states map naturally to the binary digits 1 and 0.

Advantages of binary for computers:

- **Simple hardware:** Only two states to distinguish (easier than 10 states for denary)
- **Reliable:** Easy to detect whether a signal is "on" or "off" even with electrical noise
- **Fast:** Simple circuits can process binary very quickly
- **Universal:** All data (text, images, sound) can be converted to binary

The trade-off:

Binary numbers require more digits than denary to represent the same value. For example:

- Denary 255 = Binary 11111111 (8 digits)
- Denary 1000 = Binary 1111101000 (10 digits)

However, the simplicity and reliability of binary circuits more than compensates for the extra digits.

1.1 Binary to denary conversion

To convert binary to denary, multiply each binary digit by its place value and add the results together.

Method:

1. Write out the place values above each binary digit (128, 64, 32, 16, 8, 4, 2, 1)
2. For each position with a 1, note the place value
3. Add all the place values together

Example: Convert 10110101_2 to denary

128	64	32	16	8	4	2	1
1	0	1	1	0	1	0	1

= 128 + 32 + 16 + 4 + 1 = **181**

> **Exam tip:** Always write out the place values — it prevents mistakes and shows your working for marks.

1.2 Denary to binary conversion

To convert denary to binary, use repeated subtraction of the largest possible power of 2.

Method:

1. Start with 128. Can it fit? If yes, write 1 and subtract. If no, write 0.
2. Move to 64, then 32, 16, 8, 4, 2, 1 — repeating the process
3. Continue until you reach 0

Example: Convert 201 to 8-bit binary

201 − 128 = 73 → write **1**
73 − 64 = 9 → write **1**
9 − 32? No → write **0**
9 − 16? No → write **0**
9 − 8 = 1 → write **1**
1 − 4? No → write **0**
1 − 2? No → write **0**
1 − 1 = 0 → write **1**
Answer: **11001001_2**

1.3 Understanding binary place values

Each position in a binary number represents a power of 2. The rightmost bit is the **Least Significant Bit (LSB)** with place value $2^0 = 1$. The leftmost bit is the **Most Significant Bit**

(MSB).

For an 8-bit unsigned integer:

- **Minimum value:** $00000000_2 = 0$
- **Maximum value:** $11111111_2 = 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$
- **Total values:** $2^8 = 256$ different values (0 to 255)

Worked Example 1 — Binary to denary conversion

Question: Convert the binary number 11010110 to denary. Show your working. [2 marks]

Step 1: Write place values: 128, 64, 32, 16, 8, 4, 2, 1
Step 2: Identify positions with 1: 128 + 64 + 16 + 4 + 2 **(1)**
Step 3: Calculate: 128 + 64 + 16 + 4 + 2 = **214 (1)**

Exam tip: Check your answer is in range (0—255 for 8-bit). If you get 300 or a negative number, you've made an error.

Worked Example 2 — Denary to binary conversion

Question: Convert the denary number 157 to an 8-bit binary number. Show your working. [2 marks]

Step 1: 157 − 128 = 29 ✓ (write 1)
29 − 64? No (write 0)
29 − 32? No (write 0)
29 − 16 = 13 ✓ (write 1)
13 − 8 = 5 ✓ (write 1)
5 − 4 = 1 ✓ (write 1)
1 − 2? No (write 0)
1 − 1 = 0 ✓ (write 1) **(1)**
Answer: 10011101_2 **(1)**

Exam tip: Verify by converting back: 128 + 16 + 8 + 4 + 1 = 157 ✓

Worked Example 3 — Finding the range of n-bit integers

Question: State the range of values that can be stored in a 6-bit unsigned binary integer. [2 marks]

Minimum: $000000_2 = 0$ **(1)**
Maximum: $111111_2 = 32 + 16 + 8 + 4 + 2 + 1 = 63$ (or $2^6 - 1$) **(1)**

Answer: 0 to 63

Exam tip: For n-bit unsigned integers: minimum = 0, maximum = $2^n - 1$. So 6-bit = 0 to 63, 8-bit = 0 to 255.

Worked Example 4 — Calculating total possible values

Question: A computer uses 4-bit binary numbers. Calculate how many different values can be represented. [1 mark]

Formula: Total values = 2^n where n = number of bits
Calculation: $2^4 = 2 \times 2 \times 2 \times 2 =$ **16** different values **(1)**

Exam tip: Remember: $2^4 = 16$, $2^8 = 256$, $2^{16} = 65{,}536$. These powers appear frequently.

2. Hexadecimal and two's complement

Hexadecimal (base-16) uses digits 0—9 and letters A—F (representing 10—15). **Two's complement** is a method for representing negative integers in binary.

2.1 Why hexadecimal?

Hexadecimal is used because it provides a compact way to represent binary. Each hex digit represents exactly 4 binary bits, making conversions easy.

Benefits of hexadecimal:

- **Shorter:** 8 binary digits = 2 hex digits (e.g., 11111111 = FF)
- **Easier to read:** Memory addresses and colour codes are more readable
- **Less error-prone:** Fewer digits means fewer mistakes when copying
- **Direct conversion:** Each hex digit maps to exactly 4 bits

Common uses:

- Memory addresses (e.g., 0x7FFF)
- Colour codes (e.g., #FF5733 where FF=red, 57=green, 33=blue)
- MAC addresses (e.g., 00:1A:2B:3C:4D:5E)
- Error codes and debugging output
- Assembly language programming

Exam tip: When asked "why is hexadecimal used?", the key points are: (1) more compact than binary, (2) easier to read, (3) each hex digit = exactly 4 bits, making conversion simple.

2.2 Hexadecimal conversions

Binary to hexadecimal:

1. Split binary into groups of 4 bits (from the right)

2. Convert each group to its hex equivalent

> **Example: 11010110_2 → hex**
> Split: 1101 | 0110
> 1101 = 8+4+1 = 13 = D
> 0110 = 4+2 = 6 = 6
> Answer: **D6**

Hexadecimal to binary:

1. Convert each hex digit to 4-bit binary
2. Join the groups together

> **Example: $A3_{16}$ → binary**
> A = 10 = 1010
> 3 = 3 = 0011
> Answer: **10100011_2**

2.3 Two's complement for negative numbers

Two's complement allows computers to represent both positive and negative integers using the same binary addition hardware. This is more efficient than other methods like sign-magnitude (where the first bit just indicates positive or negative).

In 8-bit two's complement, the MSB (leftmost bit) represents −128 instead of +128. If the MSB is 1, the number is negative.

Why two's complement?

Two's complement has several advantages over other methods of representing negative numbers:

- **Only one zero:** Unlike sign-magnitude, there's no "negative zero"
- **Same addition hardware:** Adding positive and negative numbers works with the same circuits
- **Easy sign detection:** Just check the MSB — 1 means negative
- **Subtraction simplified:** To subtract, convert to two's complement and add

Converting negative denary to two's complement:

1. Convert the positive version to binary
2. Flip all the bits (0→1, 1→0)
3. Add 1 to the result

> **Example: Convert −45 to 8-bit two's complement**
> Step 1: +45 = 00101101
> Step 2: Flip bits = 11010010
> Step 3: Add 1 = 11010010 + 1 = **11010011**

Converting two's complement to denary:

Method 1 (place values): Use −128 for MSB, add other place values

> **Example: Convert 11010011 to denary**

= −128 + 64 + 16 + 2 + 1
= −128 + 83 = **−45**

Method 2 (reverse process): Subtract 1, flip bits, convert, add minus sign

8-bit two's complement range: −128 to +127. The MSB determines the sign: 0 = positive, 1 = negative. Zero is represented as 00000000.

Worked Example 5 — Binary to hexadecimal

Question: Convert the binary number 10111010 to hexadecimal. Show your working. [2 marks]

Step 1: Split into nibbles (4-bit groups): 1011 | 1010 **(1)**
Step 2: Convert each nibble:
1011 = 8 + 2 + 1 = 11 = B
1010 = 8 + 2 = 10 = A **(1)**
Answer: BA

Exam tip: Always split from the RIGHT. If needed, pad with leading zeros (e.g., 11010 becomes 0001 | 1010).

Worked Example 6 — Hexadecimal to denary

Question: Convert the hexadecimal number 5F to denary. Show your working. [2 marks]

Method: Multiply each digit by its place value (16^1, 16^0)
Step 1: 5 × 16 = 80 **(1)**
Step 2: F × 1 = 15 × 1 = 15
Step 3: 80 + 15 = **95 (1)**

Exam tip: Remember F = 15, E = 14, D = 13, C = 12, B = 11, A = 10. These are tested frequently.

Worked Example 7 — Negative number to two's complement

Question: Represent the denary number −73 as an 8-bit two's complement binary number. Show your working. [3 marks]

Step 1: Convert +73 to binary: 01001001 **(1)**

(64 + 8 + 1 = 73)
Step 2: Flip all bits: 10110110 **(1)**
Step 3: Add 1: 10110110 + 1 = **10110111 (1)**

Exam tip: Check: the MSB should be 1 (indicating negative). Verify by converting back: −128 + 32 + 16 + 4 + 2 + 1 = −128 + 55 = −73 ✓

Worked Example 8 — Two's complement to denary

Question: The 8-bit two's complement binary number 11100101 represents a negative integer. Convert this to denary. [2 marks]

Method: Use place values with −128 for MSB
Step 1: −128 + 64 + 32 + 4 + 1 **(1)**
Step 2: = −128 + 101 = **−27 (1)**

Exam tip: If the MSB is 1, the number is negative. Use −128 as the MSB place value for 8-bit two's complement.

3. Binary arithmetic

Binary arithmetic follows the same principles as denary arithmetic, but with only two digits (0 and 1). This section covers addition, overflow, and binary shifts — all essential topics that appear in Paper 1 every year.

Understanding binary arithmetic helps you appreciate how computers perform calculations at the hardware level. The CPU's Arithmetic Logic Unit (ALU) performs these operations millions of times per second.

3.1 Binary addition rules

Binary addition follows four simple rules:

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 10 (0, carry 1)
1 + 1 + 1 = 11 (1, carry 1)

When adding 1 + 1, the result is 2, which is 10 in binary. Write 0 and carry 1 to the next column — just like carrying in denary addition.

Example: Add 01011010 + 00110111

```
  01011010
+ 00110111
```

```
  ──────────
    10010001
```
Carries: 1 1 1 1 1 0 0

Check: 90 + 55 = 145 ✓

3.2 Overflow

Overflow occurs when the result of a calculation is too large to fit in the available number of bits. For 8-bit unsigned integers, overflow occurs if the result exceeds 255.

How to detect overflow:

- If there is a carry OUT of the MSB position, overflow has occurred
- The result wraps around (e.g., 200 + 100 = 300, but 300 − 256 = 44 is stored)

Consequences of overflow:

- The stored value is incorrect
- Programs may produce unexpected results
- Critical in safety systems and financial calculations

Example of overflow:

```
    11010110  (214)
  + 01100100  (100)
  ──────────
  1 00111010  (overflow!)
```

Result should be 314, but only 00111010 (58) is stored.

Overflow detected: carry out of bit 7.

3.3 Binary shifts

A **binary shift** moves all bits left or right by a specified number of positions. Shifts provide a fast way to multiply or divide by powers of 2. In hardware, shift operations are much faster than multiplication or division operations.

Left shift

Moves bits to the LEFT. Zeros fill in on the right. Bits that shift beyond the MSB are lost.

Example: Left shift 00010110 by 2 places

Before: 00010110 (= 22)

After: 01011000 (= 88)

Effect: $22 \times 2^2 = 22 \times 4 = 88$ ✓

Effect: Multiply by 2^n where n = shift amount

Risk: If the MSB becomes 1 or bits are lost, the result may be incorrect (similar to overflow).

Right shift

Moves bits to the RIGHT. Zeros fill in on the left. Bits that shift beyond the LSB are lost.

Example: Right shift 01011000 by 2 places

Before: 01011000 (= 88)

After: 00010110 (= 22)

Effect: $88 \div 2^2 = 88 \div 4 = 22$ ✓

Effect: Integer divide by 2^n (remainder lost)
Note: This is a logical right shift — zeros always fill from the left.

Understanding shift operations step by step

Let's trace through a left shift operation in detail:

Left shift 00110100 by 1 place:

```
Original:  0 0 1 1 0 1 0 0  (= 52)
Shift ←1:  0 1 1 0 1 0 0 0  (0 added on right)
Result:    0 1 1 0 1 0 0 0  (= 104)
```

Check: $52 \times 2 = 104$ ✓

Right shift 00110100 by 1 place:

```
Original:  0 0 1 1 0 1 0 0  (= 52)
Shift →1:  0 0 0 1 1 0 1 0  (0 added on left, rightmost bit lost)
Result:    0 0 0 1 1 0 1 0  (= 26)
```

Check: $52 \div 2 = 26$ ✓

Important: Right shift performs integer division — any remainder is discarded. For example, 01010101 (85) shifted right by 1 gives 00101010 (42), not 42.5. The 0.5 is lost.

Worked Example 9 — Binary addition

Question: Add the binary numbers 01101011 and 00101110. Show your working. [2 marks]

```
  01101011  (107)
+ 00101110  ( 46)
 ----------
  10011001  (153)
```

Working: Add column by column from right, carrying when sum ≥ 2 **(1)**
Carries: 1 1 1 1 0 0 0
Answer: 10011001 **(1)**
Verification: 107 + 46 = 153 ✓

Exam tip: Write carries above the columns clearly. Check by converting both operands and result to denary.

Worked Example 10 — Addition with overflow detection

Question: Add the 8-bit binary numbers 11001010 and 01110010. State whether overflow has occurred and explain your answer. [3 marks]

```
  11001010  (202)
+ 01110010  (114)
──────────
1 00111100  (overflow!)
```

Calculation: Result = 00111100 with carry out **(1)**
Overflow: Yes, overflow has occurred **(1)**
Explanation: There is a carry out of the MSB (bit 7), meaning the result (316) exceeds the maximum 8-bit value of 255 **(1)**

> **Exam tip:** If a question asks to "state whether overflow occurred", you MUST explain HOW you know (carry out of MSB, or result > 255).

Worked Example 11 – Left binary shift

Question: The binary number 00010101 is shifted left by 3 places. Give the result in binary and explain the effect on the denary value. [3 marks]

Original: 00010101 = 16 + 4 + 1 = 21
Shift left by 3: Move all bits 3 places left, add zeros on right
Result: 10101000 **(1)**
New value: 128 + 32 + 8 = 168 **(1)**
Effect: The value is multiplied by 2^3 = 8. Original 21 × 8 = 168 ✓ **(1)**

> **Exam tip:** Show the multiplication: shift left by n = multiply by 2^n. Always verify with the actual values.

Worked Example 12 – Right binary shift with data loss

Question: The binary number 01100111 is shifted right by 2 places. Give the result in binary and state the denary values before and after the shift. Explain any data loss. [4 marks]

Original: 01100111 = 64 + 32 + 4 + 2 + 1 = 103 **(1)**
Shift right by 2: Move all bits 2 places right, add zeros on left
Result: 00011001 **(1)**
New value: 16 + 8 + 1 = 25 **(1)**
Expected: 103 ÷ 4 = 25.75, but result = 25
Data loss: The rightmost 2 bits (11) were lost, causing the remainder (0.75 or 3 in binary) to be discarded. Integer division only. **(1)**

> **Exam tip:** Right shifts cause data loss when bits "fall off" the right end. This is integer division — the remainder is always lost.

Worked examples summary

The table below summarises the 12 worked examples covered in this topic, organised by section and difficulty level.

#	Topic	Question type	Marks	Key skill
1	Binary → denary	Convert with working	2	Place value addition
2	Denary → binary	Convert with working	2	Repeated subtraction method
3	Binary range	State range for n-bit	2	Apply 0 to 2^n-1 formula
4	Total values	Calculate 2^n	1	Powers of 2
5	Binary → hex	Convert via nibbles	2	Split into 4-bit groups
6	Hex → denary	Convert with place values	2	×16, ×1 method
7	Negative → two's comp	Three-step conversion	3	Flip + add 1
8	Two's comp → denary	Use −128 place value	2	MSB as negative
9	Binary addition	Add with carries	2	Column addition, carries
10	Overflow detection	Add and identify overflow	3	Detect carry out of MSB
11	Left shift	Shift and explain effect	3	$\times 2^n$ relationship
12	Right shift + loss	Shift and explain data loss	4	$\div 2^n$ with remainder loss

Quick reference

Key terms and definitions

Term	Definition (mark-scheme style)
Binary	Base-2 number system using digits 0 and 1
Denary	Base-10 number system using digits 0—9 (also called decimal)

Term	Definition (mark-scheme style)
Hexadecimal	Base-16 number system using digits 0—9 and letters A—F
Bit	A single binary digit (0 or 1); smallest unit of data
Nibble	4 bits; represents one hexadecimal digit
Byte	8 bits; can represent values 0—255 (unsigned)
MSB	Most Significant Bit; the leftmost bit with highest place value
LSB	Least Significant Bit; the rightmost bit with place value 1
Unsigned integer	A whole number that can only be positive or zero
Signed integer	A whole number that can be positive, negative, or zero
Two's complement	Method of representing signed integers where the MSB has a negative place value
Overflow	Error when a calculation result exceeds the maximum storable value
Binary shift	Moving all bits left or right by a number of positions
Left shift	Shift bits left; multiplies value by 2^n
Right shift	Shift bits right; integer divides value by 2^n

Conversion quick reference

Conversion	Method
Binary → Denary	Add place values where bit = 1 (128, 64, 32, 16, 8, 4, 2, 1)
Denary → Binary	Subtract largest power of 2 that fits, repeat
Binary → Hex	Split into nibbles (4 bits), convert each to hex digit
Hex → Binary	Convert each hex digit to 4-bit binary, join
Hex → Denary	First digit × 16 + second digit × 1
Denary → Hex	Divide by 16, remainder is second digit, quotient is first
Positive → Two's comp	Same as unsigned binary (MSB will be 0)
Negative → Two's comp	Convert positive, flip all bits, add 1
Two's comp → Denary	If MSB=1: use −128 as MSB place value, add rest

8-bit ranges

Type	Minimum	Maximum	Total values
8-bit unsigned	0	255	256 (2^8)
8-bit two's complement	−128	+127	256 (2^8)

Typical pitfalls

Pitfall 1 — Wrong place values.
Wrong: Using 256, 128, 64... (starting too high) or 1, 2, 4... from the left.
Right: 8-bit place values from LEFT: 128, 64, 32, 16, 8, 4, 2, 1. The MSB is 128, not 256.

Pitfall 2 — Forgetting carries in addition.
Wrong: 1 + 1 = 2 or 1 + 1 = 1.
Right: 1 + 1 = 10 in binary (write 0, carry 1). Don't forget 1 + 1 + 1 = 11 (write 1, carry 1).

Pitfall 3 — Hex letters wrong.
Wrong: F = 16 or A = 1.
Right: A=10, B=11, C=12, D=13, E=14, F=15. The highest single hex digit is F (15), not 16.

Pitfall 4 — Splitting nibbles wrong.
Wrong: Splitting 11010 as 110 | 10.
Right: Split from the RIGHT: 0001 | 1010. Pad with leading zeros if needed.

Pitfall 5 — Two's complement method error.
Wrong: Flip bits then forget to add 1.
Right: Three steps: (1) Convert positive to binary, (2) Flip ALL bits, (3) Add 1. All three steps are required.

Pitfall 6 — Wrong sign detection.
Wrong: "Any binary with 1s is negative."
Right: In two's complement, ONLY the MSB determines sign. If MSB = 1, negative. If MSB = 0, positive (or zero).

Pitfall 7 — Confusing overflow.
Wrong: "Overflow means the answer is wrong."

Right: Overflow means the result doesn't fit in the available bits. The calculation itself was correct, but the stored value is incorrect due to the lost carry bit.

Pitfall 8 — Shift direction confusion.
Wrong: Right shift multiplies, left shift divides.
Right: LEFT shift = MULTIPLY by 2^n. RIGHT shift = DIVIDE by 2^n. Think: left makes numbers bigger (more significant).

Pitfall 9 — Forgetting right shift data loss.
Wrong: 85 >> 1 = 42.5.
Right: Binary shifts perform INTEGER operations. 85 >> 1 = 42 (remainder lost). Bits that fall off are gone forever.

Pitfall 10 — Not showing working.
Wrong: Just writing the answer "214".
Right: Show place values, show additions/subtractions. Even a correct answer may lose marks without working in multi-mark questions.

Assessment Hub | Long-Answer | Notes Index

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.